CS-412 Software Security
Fuzzing lab
Group 17

Arthur Wolf Sotero Pedro Romero Morón Mikhaïl Perevoznyk

Eric Dall'Acqua

Link to the repository:
`https://github.com/SoteroR/SoftSec2026-Fuzzing`

Questions

1. SDL is a huge library with a lot of very different functions that go from image processing, to game controllers and audio settings. As both other libraries were focused on images, we wanted to focus on something else. We finally decided to fuzz audio related function for various reasons. One important factor to consider, is that we like music, and the topic was not new to us. Also, it aligned well with what we learned in class, as a random stream of bits can be interpreted as a music pattern.

   We also considered to fuzz the image functions or the video functions. However, these areas feel very similar to what everyone was doing, and it did not totally convince us.

   Our initial approach was to fuzz the loading function for .wav files. This approach was basic and gave us little coverage, moreover, we were very limited by the wav formatting. Nevertheless we were able to find a seed that crashed during the audio build up.

   We then evolved and transitioned to a common audio work-path. Load an audio file, change its format, mix it, change it volume and stream it. This approach gave us a higher coverage and gave us more flexibility as we were not using .wav files anymore.

   Detailed harness explanations can be found in the code comments.

2. During the compilation of the library, these flags were used:

   `-fsanitize=address / AFL_USE_ASAN=1`: This flag enables the address sanitizer instrumentation within the library. This instrumentation is used to detect some bugs as early as possible. Specifically, it focuses on buffer overflows, a little of use-after-free and some spatial and temporary memory safety violations. To achieve this, it delimitates the different memory regions with red zones to which writing is not allowed, depending if it is on globals, heap or stack, these red regions are stored differently. It mainly uses a shadow memory region that maps 8 bytes into 1 byte.

   `-O1`: This flag refers to the amount of optimization we want to apply to the program, higher optimization will run it faster but will make debugging it more difficult. We decided to stick with "`-O1`" as it is a good balance between speed and debug-ability.

   `-g`: This flag just adds debugs symbols, which, as the description says, helps us debug.

3. Our initial seed corpus consists of 10 small binary files designed specifically for our harness. As explained in question 1, the harness does not parse WAV files. Instead, it interprets the input as a structured byte string where the first bytes select audio conversion parameters and the remaining bytes are treated as raw audio payload. Therefore, we use .bin seeds rather than .wav seeds.

   More formally, each input is a byte string and the harness interprets the prefix of each string as a control header:

   - data[0] → source frequency index
   - data[1] → destination frequency index
   - data[2] → source audio format index
   - data[3] → destination audio format index
   - data[4] → source channel count
   - data[5] → destination channel count
   - data[6] → payload offset
   - data[7] → conversion-shape bit flag

   The suffix is then used as the audio payload. Our seeds cover the main classes induced by this harness (from 1 to 10 as in the `/seeds` folder: no-op conversion, format conversion, integer-to-float conversion, endian conversion, channel expansion, channel reduction, frequency upsampling, frequency downsampling, extreme downsampling, combined format/channel/frequency change, and non-default payload offset)

These seeds deliberately reach different equivalence classes of the SDL audio conversion behavior and are therefore better than random bytes or a specific existing file format.

Our dictionary `sdl.dict` provides terminal strings over the alphabet of the input byte strings, which are lexical tokens for the input language accepted by our harness. It contains three main categories:

- Control byte tokens (\x00, \x01, ..., \x0d): These are useful because the first bytes of the input are interpreted modulo the number of supported formats, frequencies and channels and they directly select different entries in the harness's format and frequency arrays.
- Conversion-shape flags (\x00, \x01, \x02, \x04, \x07): These target `data[7]` which is used as a bitmask in the harness.
- Audio boundary tokens (\x00, \xff, \x80, \x7f, 16-bit and 32-bit min/max values, float values): These are useful in the payload part of the input because boundary values are more likely to trigger edge cases in conversions and audio mixing.

From the AFL++ status screen (Figure 1), we can quantify the contribution of each mutation strategy to path discovery. Bit-flip mutations contributed 4 new paths in total, arithmetic mutations contributed 2, known integer substitutions contributed 1, and dictionary-based mutations contributed 9 new paths. Dictionary-based mutations also appeared to be as much as 3 times higher in terms of contributed new paths in other runs (for example in the runs for Q8). The largest contribution came from the havoc stage, which discovered 824 new paths. On the other hand, byte-flip mutations and splice mutations did not contribute any new paths during this campaign.

This suggests that our harness benefits significantly from large random combined mutations (havoc), while the dictionary also proved useful because the input format contains structured control bytes interpreted semantically by the harness.

4. The AFL++ status screen (Figure 1) at the end of the run reports a stability of 100.00%, a corpus count of 958, a map density of 2.66%, and a total of 10 cycles done. The run lasted 36 min 55 sec, with 3 saved crashes and 36 saved hangs.

   The `afl-plot` edges graph (Figure 2) shows an exponential discovery curve at the beginning that seems to reach a limit after a few minutes. The curve rose quickly during the first few minutes, reaching about 880 edges by t = 200s. After that, edge discovery slowed abruptly. Indeed, it only increased with difficulty to 890 at 900s. From this moment on, the edge count stayed flat at this value until the end of the run.

   This suggests the campaign reached coverage saturation for this configuration. The strongest evidence is that no new edges were found for the final 20 min of the run, even though AFL++ completed several more queue cycles, ending at 10 cycles total.

   Yet, the last crash was discovered 8 min after the last edge was found. This gives us the insight that even if coverage is not growing, fuzzing can still find crashes and that finishing a fuzzing campaign earlier just based on coverage is not necessarily a good choice.

5. We had a total of `13` crashes with our different campaigns. The one we picked is a heap buffer overflow, and Figure 3 shows the ASan stack trace generated when triggering the bug after minimizing it and reproducing it. The input that causes this crash can be found in `/minimized_crash/minimized_crash.bin`, alongside the inputs that cause the two other crashes we found.

   In order to minimize the crash, we used the following command: "afl-tmin -i crashing_input -o minimized_input – ./target @@". The command used to reproduce it was "./target-afl minimized_crash/minimized_crash.bin" (Figure 3)

The crash occurred while adding data to the stream, likely due to an incorrect length or index calculation within the function itself. Further investigation is required to determine the exact root cause; however, this is outside the scope of the project.

6. Two real world applications that use SDL are SameBoy and PPSSPP.

   SameBoy is a Game Boy / Game Boy Color emulator that is written in C that uses a nativie Cocoa frontend on macOS and and SDL frontend for other platforms. A concrete attack scenario would be an attacker distributing a malicious Game Boy ROM which the user would open in the SDL frontend of SameBoy. The ROM controls the emulated audio hardware behavior and can produce unusual audio patterns, timings, or buffer sizes. If a bug exists in SDL's audio conversion pipeline, for example in `SDL_BuildAudioCVT` or `SDL_ConvertAudio`, malformed or unexpected conversion parameters could cause memory corruption, crash, or potentially code execution in the emulator process. This matters because emulators often run untrusted ROMs downloaded from the internet.

   PPSSPP is a PSP emulator available on Windows, Linux, macOS, Android, and iOS. It allows users to run PSP games from `.ISO` or `.CSO` files. The SDL frontend is present in the PPSSPP source tree and includes SDL headers and input, audio, and windowing integration. A concrete attack scenario would be for an attacker to publish a malicious crafter ISO or CSO image of a PSP game. When opened in PPSSPP's SDL frontend, the emulator would process attacker-controlled game data. Like for SameBoy, if SDL's audio conversion code mishandles edge cases, the malicious game could trigger a denial of service or memory safety bug through the emulator.

   Our harness mainly targets functions related to initializing and cleaning up SDL resources, as well as handling audio conversion logic, and therefore it doesn't cover how applications like emulators use SDL as a full platform layer. Two code paths that are not exercised by our harness would therefore be video, rendering, window and texture paths on the one hand, and input and event handling on the other hand. Concerning the first gap we identified, this could matter in the context of emulators because video output is one of SDL's main use cases, using functions such as `SDL_CreateWindow`, `SDL_CreateRenderer`, or `SDL_CreateTexture`. A malicious ROM may influence frame dimensions, pixel formats, shader usage, etc through the emulator. Even if the audio conversion code is safe, vulnerabilities could exist in these part of the library, and our harness would not detect them. For the second gap we mentioned, it could matter in this context because emulators rely heavily on SDL for keyboard, gamepad and joystick support regarding controller inputs with functions like `SDL_PollEvent`. The attacker could try to use malformed controller mappings or platform-specific input backends to exploit bugs our harness would still not have caught given its focus on audio-related functions.

7. The results are different mainly because the two campaigns collect coverage in different ways.

   With compile-time instrumentation, AFL++ inserts coverage-tracking code directly into the program when it is compiled. This is fast, so the fuzzer can execute many more inputs per second. In our results, the instrumented campaign reached 316.01 exec/s, found 892 edges, and produced a corpus of 958 inputs.

   In QEMU mode, the binary is not instrumented. AFL++ runs it through QEMU, which dynamically translates the program while it executes and collects coverage from the translated basic blocks. This adds much more overhead, so the campaign is slower. In our results, QEMU reached only 26.24 exec/s, found 126 edges (Figure 5), and produced a corpus of 15 inputs (Figure 4).

   Because QEMU mode executes fewer test cases in the same wall-clock time, it has fewer chances

to discover new paths. This explains why it found fewer edges and added fewer interesting inputs to the corpus. Also, QEMU coverage is recovered dynamically from the binary, so it may not match compile-time edge instrumentation exactly.

Overall, the instrumented campaign performs better because its coverage collection is faster and more precise, while QEMU mode is slower but useful when the source code or an instrumented binary is not available.

8. The number of instrumented edges reported by AFL during the target library compilation was of 35779. The number of edges reported by AFL during fuzzing for the harness binary was of 33594. There is a difference of around 2000 edges.

   There are some reasons for this. When the harness is compiled and linked with the binary, unreachable paths are removed. To test this even more, we made a dummy harness that just imported the file. The total edges reported in this case was of 7. Supporting our statement that unreachable edges are removed, and therefore explaining why our harness binary has less edges than the targetted library.

   Regarding map density, we achieved a density of 2.66% in the same run mentioned above. This represent the percentage of edges we achieved to traverse.

   SDL2 is a very diverse library. As we only focused on a tiny part (the audio-related functions) it is reasonable that our map density is that low. To increase it we would need to fuzz other categories like images or videos.

   Exec speeds measures have been done under three configurations:

   - No sanitizer + fork mode: 430.57 exec/s
   - ASan + fork mode: 316.01 exec/s
   - ASan + persistent mode: 817.93 exec/s

   The basic version is faster than the ASan fork-mode version because it has no sanitizer checks. In the ASan version, AddressSanitizer adds extra runtime checks for memory errors, such as out-of-bounds accesses and use-after-free bugs. These checks are useful for finding bugs, but they add overhead, so the speed drops from 430.57 exec/s to 316.01 exec/s.

   The persistent ASan version is the fastest, even though ASan is enabled. This is because persistent mode avoids restarting the whole target process for every test case. In normal fork mode, AFL++ has to fork a new process for each input, which costs time. In persistent mode, the program stays alive and runs many inputs inside the same process, reducing startup and fork overhead.

   ASan slows the program down because of extra memory checks, while persistent mode speeds it up because it reuses the same process across many test cases. This is why the persistent + ASan configuration reaches 817.93 exec/s, which is faster than both fork-mode configurations.

Appendix

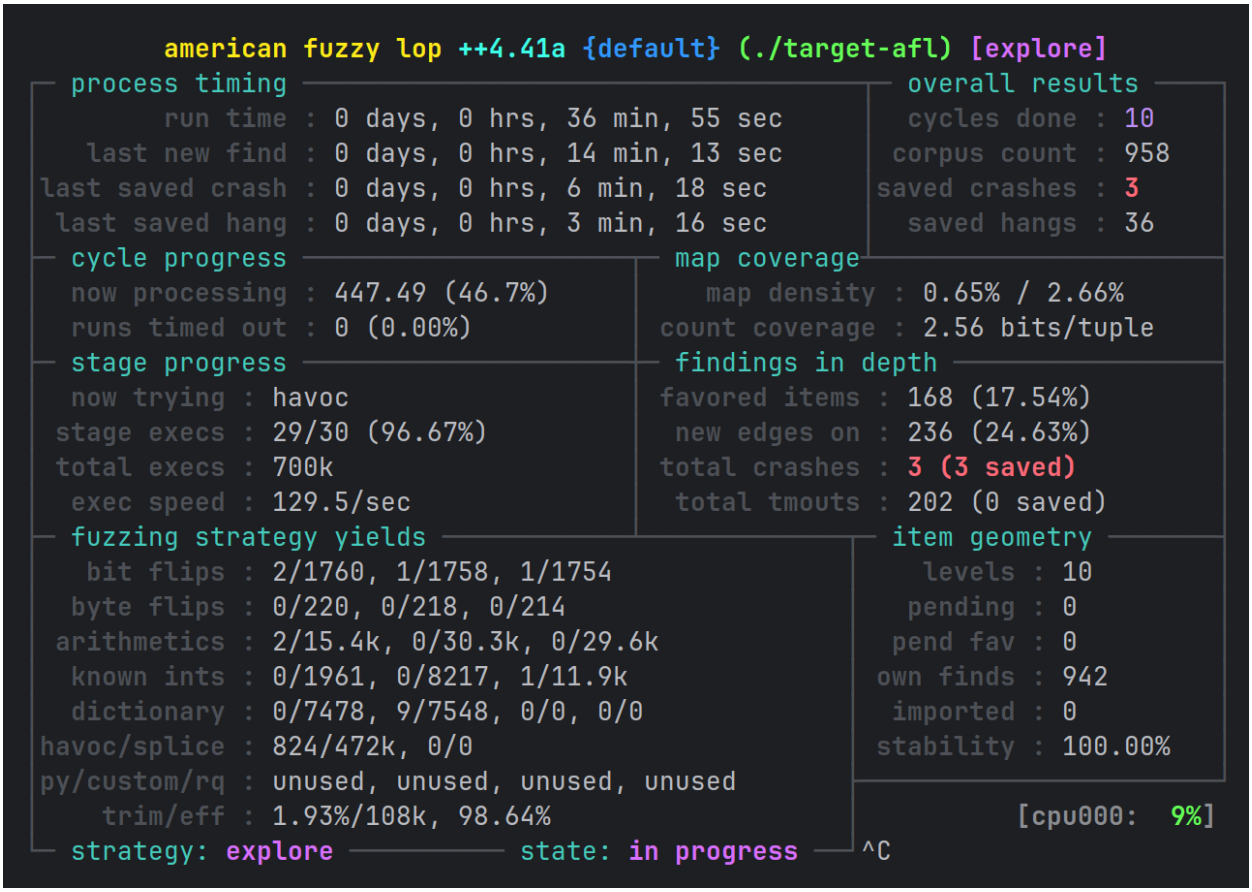

Figure 1: AFL++ status screen for ASan + no persistent mode

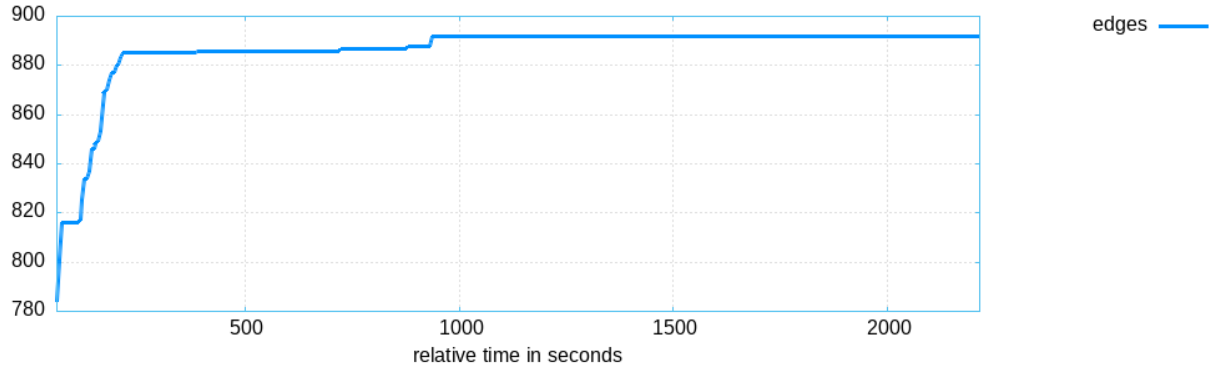

Figure 2: Edges graph for ASan + no persistent mode

```
[AFL++ a32e2e270963] /app # ./target-afl minimized_crash/minimized_crash.bin
=================================================================
==137==ERROR: AddressSanitizer: heap-buffer-overflow on address 0x7afe64cf9000 at pc 0x55669e4417e8 bp 0x7ffd29751eb0 sp 0x7ffd29751668
READ of size 393216000 at 0x7afe64cf9000 thread T0
    #0 0x55669e4417e7 in memcpy (/app/target-afl+0xdb7e7) (BuildId: 6533e4d3d19441a97bafcb3df2701d30a57f60a0)
    #1 0x55669e58f0a0 in SDL_AudioStreamPutInternal /tmp/SDL2-2.30.2/src/audio/SDL_audiocvt.c:1207:9
    #2 0x55669e58ea25 in SDL_AudioStreamPut_REAL /tmp/SDL2-2.30.2/src/audio/SDL_audiocvt.c:1294:20
    #3 0x55669e484755 in fuzz_one_input /app/src/harness.c:206:9
    #4 0x55669e483fac in main /app/src/harness.c:286:9
    #5 0x7afeadfab1c9  (/lib/x86_64-linux-gnu/libc.so.6+0x2a1c9) (BuildId: 8e9fd827446c24067541ac5390e6f527fb5947bb)
    #6 0x7afeadfab28a in __libc_start_main (/lib/x86_64-linux-gnu/libc.so.6+0x2a28a) (BuildId: 8e9fd827446c24067541ac5390e6f527fb5947bb)
    #7 0x55669e3a29f4 in _start (/app/target-afl+0x3c9f4) (BuildId: 6533e4d3d19441a97bafcb3df2701d30a57f60a0)

0x7afe64cf9000 is located 6144 bytes before 393216068-byte region [0x7afe64cfa800,0x7afe7c3fa844)
allocated by thread T0 here:
    #0 0x55669e442c40 in realloc (/app/target-afl+0xdcc40) (BuildId: 6533e4d3d19441a97bafcb3df2701d30a57f60a0)
    #1 0x55669e50e901 in SDL_realloc_REAL /tmp/SDL2-2.30.2/src/stdlib/SDL_malloc.c:5326:11

SUMMARY: AddressSanitizer: heap-buffer-overflow (/app/target-afl+0xdb7e7) (BuildId: 6533e4d3d19441a97bafcb3df2701d30a57f60a0) in memcpy
Shadow bytes around the buggy address:
  0x7afe64cf8d80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0x7afe64cf8e00: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0x7afe64cf8e80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0x7afe64cf8f00: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0x7afe64cf8f80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
=>0x7afe64cf9000:[fa]fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
  0x7afe64cf9080: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
  0x7afe64cf9100: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
  0x7afe64cf9180: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
  0x7afe64cf9200: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
  0x7afe64cf9280: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
Shadow byte legend (one shadow byte represents 8 application bytes):
  Addressable:           00
  Partially addressable: 01 02 03 04 05 06 07
  Heap left redzone:       fa
  Freed heap region:       fd
  Stack left redzone:      f1
  Stack mid redzone:       f2
  Stack right redzone:     f3
  Stack after return:      f5
  Stack use after scope:   f8
  Global redzone:          f9
  Global init order:       f6
  Poisoned by user:        f7
  Container overflow:      fc
  Array cookie:            ac
  Intra object redzone:    bb
  ASan internal:           fe
  Left alloca redzone:     ca
  Right alloca redzone:    cb
==137==ABORTING
[AFL++ a32e2e270963] /app #
```

Figure 3: ASan stack trace for one of the crashes found by the campaign

```
      american fuzzy lop ++4.41a {default} (./target-normal) [explore]
┌─ process timing ──────────────────────────────────────┬─ overall results ─────┐
│        run time : 0 days, 0 hrs, 30 min, 1 sec        │  cycles done : 24     │
│   last new find : 0 days, 0 hrs, 16 min, 6 sec        │ corpus count : 15     │
│last saved crash : none seen yet                       │saved crashes : 0      │
│ last saved hang : 0 days, 0 hrs, 16 min, 8 sec        │  saved hangs : 8      │
├─ cycle progress ────────────────────┬─ map coverage ─┴───────────────────────┤
│  now processing : 10.32 (66.7%)    │    map density : 0.07% / 0.19%         │
│  runs timed out : 0 (0.00%)        │ count coverage : 1.00 bits/tuple       │
├─ stage progress ────────────────────┼─ findings in depth ────────────────────┤
│  now trying : havoc                │ favored items : 8 (53.33%)             │
│ stage execs : 35/112 (31.25%)      │  new edges on : 10 (66.67%)            │
│ total execs : 43.5k                │ total crashes : 0 (0 saved)            │
│  exec speed : 101.6/sec            │  total tmouts : 631 (0 saved)          │
├─ fuzzing strategy yields ───────────┴───────────────┬─ item geometry ───────┤
│   bit flips : 0/0, 0/0, 0/0                         │    levels : 3         │
│  byte flips : 0/0, 0/0, 0/0                         │   pending : 0         │
│ arithmetics : 0/0, 0/0, 0/0                         │  pend fav : 0         │
│  known ints : 0/0, 0/0, 0/0                         │ own finds : 5         │
│  dictionary : 0/0, 0/0, 0/0, 0/0                    │  imported : 0         │
│havoc/splice : 4/41.8k, 0/0                          │ stability : 100.00%   │
│py/custom/rq : unused, unused, unused, unused        ├───────────────────────┘
│    trim/eff : 42.41%/1137, n/a                      │          [cpu000: 37%]
└─ strategy: explore ──────────── state: final phase ─┘^C
```

Figure 4: AFL++ status screen for QEMU + ASan

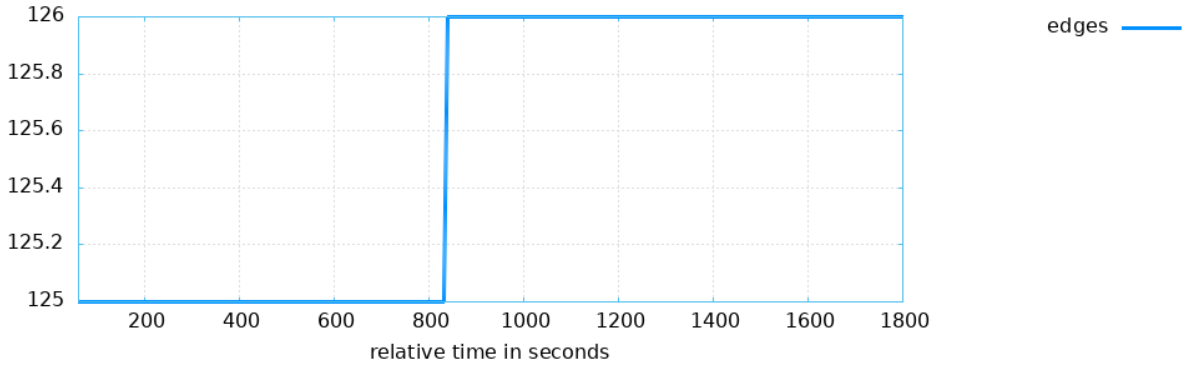

Figure 5: Edges graph for QEMU + ASan